

PHP anwenden statt nur nachschlagen

10 gemischte Aufgaben · Mittel bis anspruchsvoll

Aufgabenpool zum überarbeiteten Quick Handout: Jede Aufgabe verbindet mehrere Themen. Die Aufgaben sind einzeln auswählbar; die Zeitangaben sind Richtwerte, keine Prüfungsvorgabe.

Arbeitsweise

Schreiben Sie die benötigten Kontrollstrukturen und Funktionen selbst. Die Startdateien liefern Eingabedaten und TODO-Bereiche. Bearbeiten Sie zuerst den Pflichtteil und anschließend die Testfälle. Ergebnisse dürfen nicht fest einprogrammiert werden.

PHP-Block, strict_types und Ausgabeformat sind in den Dateien vorbereitet. Aufgaben 1–7 und 9 geben Text aus, Aufgaben 8 und 10 HTML. Für Aufgabe 10 liegt ein fertiges Formular bei. Ohne Bearbeitung kann eine Startdatei eine leere Ausgabe zeigen.

Aufgabenübersicht

01	Preisimport mit Stolperfallen	20–30 Min.
02	Anmeldeliste ohne Datenverlust	20–30 Min.
03	Konfiguration: Welche Keys überleben?	25–35 Min.
04	Fehlerjagd in einer Rabattfunktion	25–35 Min.
05	Lagerbewegungen kontrolliert verbuchen	30–40 Min.
06	Warteschlange mit begrenzten Plätzen	25–35 Min.
07	Warenkorb mit Rabatt und Versand	35–45 Min.
08	Titelsuche: Treffer an Position 0	25–35 Min.
09	Sitzplätze vergeben – ganz oder gar nicht	35–45 Min.
10	Buchungsformular prüfen und auswerten	40–55 Min.

Start unter XAMPP

Starte PHP-START.cmd im Repository-Hauptordner. Öffne im Arbeitsbereich die Grundlagenübersicht und Uebungen_Start/index.html. Dort liegen zehn PHP-Startdateien und das Formular zu Aufgabe 10.

Festlegung für Ganzzahlfelder: FILTER_VALIDATE_INT ohne Sonderflags, anschließend Bereich prüfen. Führende Nullen wie "03", Dezimalzahlen und Exponentenschreibweise sind hier keine gültigen Ganzzahlstrings. Die Preisaufgaben verwenden dagegen ausdrücklich is_numeric.

Grundlage: PHP · PHP Quick Reference, überarbeitete Ausgabe (9 Seiten). Die Abschnitt-IDs auf den Aufgaben verweisen auf das Handout.

Fachlicher Abgleich: intval · floatval · Array-Regeln · Validierung · Scope · HTML-Ausgabe

01 / Preisimport mit Stolperfallen

Mittel · 20–30 Min.

Strings · Validierung · floatval / intval · foreach

Ein Import liefert Preise in unterschiedlichen Schreibweisen. Erstellen Sie eine bereinigte Preisliste und ein Fehlerprotokoll.

Startdaten / TODO

```
$rohpreise = [  
    " 12,50 ", "8.90", "0", "1e2", "-2,50",  
    "12abc", "", " ", "1.234,56", null, ["9,90"]  
];  
  
// TODO: Prüfen, umwandeln und auswerten.
```

Arbeitsauftrag

1. Akzeptieren Sie nur Strings. Entfernen Sie äußere Leerzeichen und ersetzen Sie das Dezimalkomma durch einen Punkt. Tausendertrennzeichen werden nicht unterstützt.
2. Prüfen Sie den vollständigen bereinigten Text mit `is_numeric`. Gültig sind Preise von 0 bis 500 Euro einschließlich; der Nullpreis ist erlaubt. Die Obergrenze soll eine Konstante sein.
3. Wandeln Sie gültige Preise mit `floatval` um. Speichern Sie je Eintrag den Preis als float, den vollen Euro-Anteil als int und den auf ganze Cent gerundeten Betrag als int. Verwenden Sie dafür auch `intval`.
4. Sammeln Sie ungültige Eingaben mit Ursprungsindex und Fehlergrund. Geben Sie die gültigen Datensätze, beide Anzahlen und die Summe in Cent aus.

Testfälle

Zusätzlich: "500", "500,01", "12,345", "0,10" und ein tatsächlicher int-Wert 12. Prüfen Sie auch eine vollständig leere Importliste.

Kontrollfrage / Zusatz

Warum darf die Validierung nicht erst nach `intval` oder `floatval` stattfinden? Wie behandelt Ihre Lösung "1e2"?

Abgabe: Bereinigte Datensätze + Fehlerliste + Anzahl gültig/ungültig + Summe in Cent.

Im Handout: 02-05 bis 02-08 · 03-05 · 09-06

02 / Anmeldeleiste ohne Datenverlust

Mittel · 20–30 Min.

explode / implode · eigene Funktionen · Arrays · Dubletten

Die Teilnehmer-Kürzel stammen aus einer durch Semikolon getrennten Textzeile. Das Kürzel "0" ist ausdrücklich gültig.

Startdaten / TODO

```
$rohtext = " Mia ; TOM; ; mia; 0; Lena ; tom; ";

// TODO: Bereinigungsfunktion(en) selbst schreiben.
// TODO: Importieren, zählen und sortiert ausgeben.
```

Arbeitsauftrag

1. Zerlegen Sie den Text am Semikolon. Normalisieren Sie jedes Kürzel: äußere Leerzeichen entfernen und ASCII-Buchstaben kleinschreiben. Verwenden Sie dafür eine eigene Funktion mit Parametertyp und Rückgabotyp.
2. Entfernen Sie nur Kürzel, die nach der Bereinigung aus keinem Zeichen mehr bestehen. Entfernen Sie anschließend doppelte Werte; der erste Eintrag soll erhalten bleiben.
3. Erzeugen Sie eine aufsteigend sortierte Liste mit lückenlosen numerischen Keys. Geben Sie die Kürzel zusätzlich als einen Text mit dem Trenner " | " aus.
4. Zählen Sie getrennt: ursprüngliche Felder, entfernte Leerfelder, entfernte Dubletten und verbleibende Kürzel. Kontrollieren Sie, dass die Zahlen zusammenpassen.

Testfälle

Wiederholen Sie den Import mit "0;0", "; ;", "MIA;mia;Mia" und "". Für einen leeren Gesamttext gelten die beim Zerlegen tatsächlich entstehenden Felder als Ausgangsbasis.

Kontrollfrage / Zusatz

An welcher Stelle würde ein pauschales empty oder array_filter ohne eigene Prüfregel ein gültiges Kürzel verlieren?

Abgabe: Sortierte Liste + verbundener Text + vier nachvollziehbare Zähler.

Im Handout: 06-05 · 07-02 bis 07-06 · 08-05 · 09-04

03 / Konfiguration: Welche Keys überleben?

Mittel bis anspruchsvoll · 25–35 Min.

Key-Typen · doppelte Keys · array_merge / + · null

Zwei Konfigurationsquellen enthalten überlappende und ungewöhnliche Schlüssel. Arbeiten Sie zunächst als Schreibtischtest, danach mit PHP.

Startdaten / TODO

```
$basis = ["modus" => "standard", "limit" => "5", 0 => "Start"];
$import = [
    "modus" => "kurs", "limit" => null,
    "8" => "Text-8", 8 => "Zahl-8", "08" => "Null-8",
    "" => "Leer", null => "Nullkey"
];

// TODO: Zuerst Vermutungen notieren, dann pruefen.
```

Arbeitsauftrag

1. Notieren Sie vor dem Ausführen alle tatsächlich verbleibenden Einträge in \$import: Reihenfolge, Key, Key-Typ und Wert. Erklären Sie jeden weggefallenen Eintrag.
2. Erzeugen Sie zwei neue Zusammenführungen aus \$basis und \$import: einmal mit array_merge und einmal mit +. Vergleichen Sie Werte, Anzahl und numerische Keys; verändern Sie die Originale nicht.
3. Untersuchen Sie in \$import die Keys "limit", "fehlt" und "" mit isset, array_key_exists und empty. Vergleichen Sie außerdem beide Ersatzwert-Operatoren bei "limit".
4. Löschen Sie in einer Kopie den Integer-Key 8 und hängen Sie mit leeren eckigen Klammern einen neuen Wert an. Bestimmen Sie dessen Key. Erzeugen Sie danach eine Version ohne Indexlücken.

Testfälle

Ergänzen Sie "Name" und "name" als verschiedene Keys. Ersetzen Sie testweise den Wert von "limit" durch "0" und vergleichen Sie die Prüfungen erneut.

Kontrollfrage / Zusatz

Warum reichen print_r und count allein nicht aus, um Key-Typen und den nächsten automatischen Key sicher zu beurteilen?

Abgabe: Vorhersage als Kommentar + überprüfte Array-Ausgaben + kurze Begründungen.

Im Handout: 05-02 bis 05-06 · 06-08 · 09-03 bis 09-05

04 / Fehlerjagd in einer Rabattfunktion

Mittel bis anspruchsvoll · 25–35 Min.

Bezeichner · Scope · Typen · Zuweisung / Vergleich · PHPDoc

Der folgende Code soll einen Rabattpreis berechnen, enthält aber Sprachfehler und Logikfehler. Er liegt absichtlich im TODO-Kommentar.

Startdaten / TODO

```
$Endpreis = 100.0;
/* TODO: Diesen fehlerhaften Code bearbeiten.
const maxRabatt = 20;
function BerechnePreis($2preis, $rabatt-prozent) {
    $Endpreis = $2preis;
    if ($rabatt-prozent = 10); {
        $Endpreis -= $2preis * $rabatt-prozent / 100;
    }
    echo $Endpreis;
}
$ergebnis = BerechnePreis("100", 10);
echo $Endpreis;
*/
```

Arbeitsauftrag

1. Korrigieren Sie die Bezeichner und verwenden Sie die Konventionen des Handouts. Unterscheiden Sie dabei echte Syntaxfehler von bloßen Abweichungen der Schreibweise.
2. Die Funktion erhält einen float-Preis und einen int-Rabatt. Jeder Rabatt von 0 bis zur festgelegten Obergrenze ist erlaubt. Außerhalb dieses Bereichs wird der Ausgangspreis unverändert zurückgegeben.
3. Entfernen Sie die Logikfehler. Geben Sie das Ergebnis als float zurück, gerundet auf zwei Nachkommastellen; die Funktion selbst soll nichts ausgeben. Bereiten Sie den Zahlenstring vor dem Aufruf unter `strict_types` passend vor.
4. Dokumentieren Sie die Funktion mit einem DocBlock inklusive Parameter und Rückgabe. Geben Sie danach Rückgabewert und äußere Preisvariable getrennt aus. Verzichten Sie auf global und Referenzparameter.

Testfälle

Prüfen Sie denselben Ausgangspreis mit 0, 10, 20, 21 und -1 Prozent. Kontrollieren Sie zusätzlich, ob sich die Variable außerhalb der Funktion verändert.

Kontrollfrage / Zusatz

Begründen Sie: eigener Scope bei einer Funktion, aber nicht bei `if`. Erklären Sie außerdem `//`, `#`, `/* ... */` und `/** ... */`; denken Sie beim `#`-Stil an die Sonderform `#[]`.

Abgabe: Lauffähige, dokumentierte Funktion + Testausgaben + kurze Fehlerklassifikation.

Im Handout: 01-02 bis 01-07 · 02-01 bis 02-04 · 03-01 · 04-06/07

05 / Lagerbewegungen kontrolliert verbuchen

Anspruchsvoll · 30–40 Min.

verschachtelte Arrays · FILTER_VALIDATE_INT · foreach · Kopie

Buchen Sie Lagerbewegungen in ihrer Reihenfolge. Fehlerhafte Bewegungen dürfen den Bestand nicht verändern.

Startdaten / TODO

```
$lager = [
    "A-01" => ["name" => "Stift", "bestand" => 3],
    "A-02" => ["name" => "Mappe", "bestand" => 0],
    "A-03" => ["name" => "Block", "bestand" => 10]
];
$bewegungen = [
    ["artikel" => "A-01", "aenderung" => "4"],
    ["artikel" => "A-02", "aenderung" => "-1"],
    ["artikel" => "A-01", "aenderung" => "2abc"],
    ["artikel" => "X-99", "aenderung" => "5"],
    ["artikel" => "A-03", "aenderung" => "-3"],
    ["artikel" => "A-02", "aenderung" => "0"],
    ["artikel" => "A-01", "aenderung" => []],
    ["aenderung" => "1"]
];
// TODO: Auf einer Kopie arbeiten; Fehler protokollieren.
```

Arbeitsauftrag

1. Erstellen Sie eine Arbeitskopie des Lagers. Prüfen Sie pro Bewegung: Array-Struktur, vorhandene Artikelnummer als String, bekannter Artikel und Änderung als gültiger Ganzzahlstring.
2. Buchen Sie nur, wenn der neue Bestand nicht negativ wäre. Eine Änderung um 0 ist gültig. Schreiben Sie Änderungen in den tatsächlich zugehörigen Array-Eintrag zurück.
3. Protokollieren Sie jede Bewegung mit Ursprungsindex und Status. Bei Ablehnung nennen Sie einen Grund; danach soll die Verarbeitung mit der nächsten Bewegung weitergehen.
4. Sortieren Sie die Arbeitskopie nach Artikelnummer, ohne die Zuordnung zu verlieren. Geben Sie die Bestände, ihre Gesamtsumme sowie Erfolgs- und Fehleranzahl aus. Kennzeichnen Sie Artikel mit Bestand unter 3.

Testfälle

Testen Sie außerdem eine leere Bewegungsliste und eine Abbuchung exakt in Höhe des aktuellen Bestands. Vergleichen Sie Original und Arbeitskopie.

Kontrollfrage / Zusatz

Was passiert, wenn in foreach nur die lokale Schleifenvariable geändert wird?

Abgabe: Aktualisiertes Lager + Bewegungsprotokoll + Kennzahlen; das Original bleibt erhalten.

Im Handout: 04-03/04 · 05-07 · 06-01/02/07 · 09-07

06 / Warteschlange mit begrenzten Plätzen

Anspruchsvoll · 25–35 Min.

do ... while · while · array_shift · continue · strenge Suche

Eine Folge simulierter Kapazitätseingaben steuert, wie viele Personen aus einer Warteschlange aufgenommen werden.

Startdaten / TODO

```
$kapazitaetsEingaben = ["abc", "0", "3"];
$warteschlange = [" Mia ", "", "0", "Mia", "Tom", "Lena"];

// TODO: Erst Kapazitaet ermitteln, dann Plaetze vergeben.
```

Arbeitsauftrag

1. Entnehmen Sie mit einer do-while-Schleife Kapazitätseingaben vom Anfang, bis ein gültiger Ganzzahlstring zwischen 1 und 5 gefunden wurde oder keine Eingabe mehr übrig ist. Auch eine anfangs leere Liste muss funktionieren.
2. Ohne gültige Kapazität: Meldung ausgeben und die Warteschlange unverändert lassen. Andernfalls mit einer while-Schleife Personen vom Anfang der Warteschlange entnehmen.
3. Bereinigen Sie die Kürzel mit trim. Überspringen Sie leere Werte und bereits aufgenommene Kürzel; der Vergleich ist case-sensitiv. Das Kürzel "0" darf aufgenommen werden. Ungültige Datentypen sind ebenfalls abzulehnen.
4. Beenden Sie die Aufnahme, sobald die Plätze belegt sind oder die Warteschlange leer ist. Geben Sie Aufgenommene, abgelehnte Einträge mit Grund, verbleibende Warteschlange und freie Plätze aus.

Testfälle

Kapazitätseingaben: [], ["1"], ["10", "2"]. Warteschlangen: [], ["0"] und [null, "Mia", "Mia"]. Es darf keine Endlosschleife entstehen.

Kontrollfrage / Zusatz

Welche Zustände müssen sich pro Durchlauf ändern? Warum ist do-while hier nicht einfach mit einer ungeprüften Entnahme gleichzusetzen?

Abgabe: Kapazitätsprotokoll + Platzvergabe + Restwarteschlange + freie Plätze.

Im Handout: 04-02/04 · 06-04/06 · 07-06 · 09-07

07 / Warenkorb mit Rabatt und Versand

Anspruchsvoll · 35–45 Min.

Funktionen · Preisprüfung · switch · Arrays · Cent-Rechnung

Berechnen Sie einen Warenkorb aus importierten Positionsdaten. Fehlerhafte Positionen werden protokolliert und nicht berechnet.

Startdaten / TODO

```
$positionen = [
    ["name" => " PHP-Handout ", "menge" => "2", "preis" => "12,50"],
    ["name" => "USB-Stick", "menge" => "1", "preis" => "19.90"],
    ["name" => "Sticker", "menge" => "0", "preis" => "2,00"],
    ["name" => "Mappe", "menge" => "2abc", "preis" => "4,00"]
];
$versandart = " Express ";
// TODO: Prüfen, Positionen berechnen und zusammenfassen.
```

Arbeitsauftrag

1. Prüfen Sie Name, Menge und Einzelpreis. Name: nicht leer nach trim. Menge: Ganzzahlstring von 0 bis 20. Preis: wie in Aufgabe 1, von 0 bis 500 Euro. Menge 0 ist gültig, erzeugt aber keine berechnete Position.
2. Berechnen Sie Positionen in ganzen Cent. Ab 4000 Cent Warenwert gibt es 10 % Rabatt auf die gesamte Ware. Runden Sie den Rabatt auf ganze Cent. Halten Sie Grenzen und Sätze in Konstanten fest.
3. Normalisieren Sie die Versandart und entscheiden Sie mit switch: standard kostet 490 Cent, express 990 Cent. Ab 6000 Cent Warenwert nach Rabatt ist nur Standardversand kostenlos. Andere Versandarten führen zu einer Fehlermeldung ohne Endsumme.
4. Zerlegen Sie die Verarbeitung in mindestens zwei eigene Funktionen. Geben Sie berechnete Positionen, Fehlerliste, Warenwert, Rabatt, Versand und Endsumme aus; alle Geldwerte in Cent.

Testfälle

Prüfen Sie Warenwerte knapp unter und exakt auf der Rabattgrenze sowie Standardversand unter und exakt auf der Freigrenze nach Rabatt. Testen Sie "abholung" und einen leeren Warenkorb.

Kontrollfrage / Zusatz

Legen Sie fest: Wenn keine berechnete Position übrig bleibt, lautet die Meldung "Warenkorb leer"; Versand und Endsumme werden dann nicht berechnet.

Abgabe: Positionsübersicht + Fehler + nachvollziehbare Summen oder eindeutige Abbruchmeldung.

Im Handout: 02-05 bis 02-08 · 03-03/05/07 · 04-06 · 09-07

08 / Titelsuche: Treffer an Position 0

Mittel bis anspruchsvoll · 25–35 Min.

stripes · array_search · substr · Strings · HTML-Ausgabe

Erstellen Sie eine kleine Suchausgabe für Kurstitel. Die Ausgangsdaten enthalten nur ASCII-Zeichen; eingegebene HTML-Tags sollen als Text sichtbar bleiben.

Startdaten / TODO

```
$titel = [  
    "PHP Grundlagen", " HTML und CSS ", "PHP Arrays & Strings",  
    "Java statt PHP", "<b>PHP</b> in Formularen", "SQL", ""  
];  
$suche = " php ";  
$exakterTitel = "PHP Grundlagen";  
// TODO: Suchen und Ergebnisse als sichere HTML-Liste ausgeben.
```

Arbeitsauftrag

1. Bereinigen Sie Titel und Suchtext an den Rändern. Leere Suchtexte werden mit einer Meldung abgelehnt und dürfen nicht alle Titel als Treffer liefern.
2. Suchen Sie ohne Beachtung der ASCII-Großschreibung. Geben Sie je Treffer den ursprünglichen Array-Key, die erste Fundposition und die Gesamtlänge des bereinigten Titels aus.
3. Erzeugen Sie je Treffer eine Vorschau mit höchstens 18 Originalzeichen. Nur bei längeren Titeln werden anschließend drei Punkte angehängt. Geben Sie die Vorschauen als HTML-Liste aus und maskieren Sie die Texte.
4. Suchen Sie unabhängig davon \$exakterTitel als vollständigen Wert mit array_search und strenger Typprüfung. Melden Sie entweder dessen Key oder "nicht gefunden". Ein Treffer bei Key 0 muss erkennbar sein.

Testfälle

Suchtexte: "PHP", "", " ", "Python" und "". Exakte Titel: "SQL", "Fehlt" und "PHP Grundlagen".

Kontrollfrage / Zusatz

Zusatz: Was müssten Sie bei Länge, Kürzung und Groß-/Kleinschreibung für "Käse" bzw. "MÜLLER" ändern? Benennen Sie passende Funktionen und die benötigte Erweiterung.

Abgabe: HTML-Trefferliste + Meldung für die exakte Suche; Position und Key 0 bleiben Treffer.

Im Handout: 06-06 · 07-06/07 · 08-01/03/06

09 / Sitzplätze vergeben – ganz oder gar nicht

Anspruchsvoll · 35–45 Min.

verschachtelte Schleifen · strikte Prüfung · Funktionen · Kopien

Im Sitzplan bedeutet null: frei; ein leerer String: gesperrt; jeder andere String: belegt. Auch "0" ist eine gültige Belegung.

Startdaten / TODO

```
$sitzplan = [
    "A" => ["0", "Mia", null],
    "B" => ["Tom", "", null],
    "C" => [null, "Lena", "Kai"]
];
$anzahlRoh = "3";
$gruppenname = "PHP-Team";
// TODO: Plan auswerten und Reservierung entwickeln.
```

Arbeitsauftrag

1. Zählen Sie freie, gesperrte und belegte Plätze pro Reihe und insgesamt. Verwenden Sie verschachtelte Schleifen; mindestens eine davon soll eine for-Schleife sein.
2. Validieren Sie die gewünschte Anzahl als Ganzzahlstring von 1 bis 9 und den Gruppennamen als nichtleeren String nach trim. Behalten Sie diese Prüfung getrennt von der eigentlichen Reservierungsfunktion.
3. Schreiben Sie eine Funktion mit typisierten Parametern, die eine Kopie des Plans verarbeitet. Reichen die freien Plätze nicht aus, darf kein einziger Platz verändert werden.
4. Andernfalls belegen Sie die ersten freien Plätze in Reihenfolge A, B, C und innerhalb einer Reihe von links nach rechts. Brechen Sie nach der gewünschten Anzahl ab. Liefern Sie Erfolg, neuen Plan und Sitzbezeichnungen wie A1 zurück; außen bleibt der Originalplan unverändert.

Testfälle

Testen Sie 1, 3, 4, 0 und "3abc" als gewünschte Anzahl. Geben Sie die Statuszahlen jeweils vor und nach dem Reservierungsversuch aus.

Kontrollfrage / Zusatz

Warum wäre empty für die Feststellung freier Plätze falsch? Woran erkennen Sie, dass die Rückgabe benutzt wurde und nicht versehentlich nur eine lokale Kopie verändert wurde?

Abgabe: Statuszahlen vorher/nachher + Erfolgsmeldung + Sitzbezeichnungen + neuer Plan.

Im Handout: 04-03 bis 04-07 · 05-07/08 · 06-02 · 09-04/07

10 / Buchungsformular prüfen und auswerten

Anspruchsvoll · Transfer · 40–55 Min.

GET / POST · Validierung · Arrays · switch · sichere Ausgabe

Das fertige Formular liegt als 10_formular.html bei. Beide Sende-Buttons verwenden dieselbe Auswertungsdatei – einmal POST, einmal GET. Keine Datenbank und keine Speicherung.

Startdaten / TODO

```
// Die Formulare senden diese Feldnamen:
// gast, erwachsene, kinder, naechte, tarif, extras[], Hinweis

// TODO: Request-Methode und dazugehoerige Daten ermitteln.
// TODO: Alle Felder pruefen; Fehler gemeinsam sammeln.
// TODO: Nur ohne Fehler rechnen und sicher in HTML ausgeben.
```

Arbeitsauftrag

1. Verarbeiten Sie ausschließlich GET oder POST und lesen Sie nur das dazu passende Array. Ein Aufruf ohne Formulardaten zeigt einen Hinweis statt einer Berechnung.
2. Gast: nichtleerer String nach trim. Erwachsene: Ganzzahlstring 1–4; Kinder: 0–4, nur bei fehlendem Key gilt 0; Nächte: 1–14. Erwachsene und Kinder zusammen dürfen höchstens 6 sein. Arrays, null, Leerstrings und unvollständige Zahlen sind für diese Felder ungültig.
3. Tarif: exakt standard oder komfort. Zimmerpreis je Nacht: 8000 bzw. 12000 Cent. Extras dürfen fehlen; sonst muss ein Array mit erlaubten Strings vorliegen. Erlaubt sind fruehstueck und parkplatz, jeweils höchstens einmal zu berechnen. Unbekannte Werte oder falsche Typen sind Fehler.
4. Frühstück kostet 600 Cent pro Person und Nacht, Parkplatz 800 Cent pro Nacht. Ermitteln Sie Tarifkosten mit switch und trennen Sie Prüfen, Berechnen und Ausgeben durch eigene Funktionen. Bei Fehlern alle Meldungen zeigen, aber keinen Preis berechnen.
5. Geben Sie bei Erfolg Quelle, Gast, Personenzahlen, Nächte, Tarif, Extras und Gesamtpreis aus. Ein optionaler Hinweis muss ein String sein; fehlend bedeutet leer. Zeilenumbrüche sollen sichtbar bleiben, HTML-Zeichen dürfen nicht als Markup wirken.

Testfälle

Testen Sie GET und POST; Kinder "0", fehlende Kinder, Kinder "", Nächte "2abc", unbekannten Tarif, extras als String, doppelte Extras und einen Namen wie "Mia".

Kontrollfrage / Zusatz

Die Prüfungen erfolgen auch dann, wenn das HTML-Formular bereits min, max oder required verwendet. Die simulierten fehlerhaften Requests lassen sich in einer temporären Testdatei an die Prüffunktion übergeben.

Abgabe: Gemeinsame Fehlerliste oder vollständige, sicher ausgegebene Buchungsübersicht.

Im Handout: 03-07 · 04-06 · 08-06 bis 08-08 · 09-01 bis 09-08